

Agentic Inference & Serving — Annotated Working Brief

Revised 14 Jun 2026 · a study-oriented edition with a plain-language summary of every key paper · Target: MLSys 2027 Industry Track (~Oct 2026, provisional) · public benchmarks + open infrastructure only

How to read this document

This edition is written for someone **new to the area**. Technical terms are explained in parentheses the first time they appear and collected in the **Key terms** box below. We avoid internal shorthand: where earlier drafts wrote “C1” or “H1,” this version spells out the idea (e.g., “the lead contribution” or “the interleaving hypothesis”). Each paper gets a 1–2 paragraph summary covering **what it does**, **why it matters to the field**, and a highlighted **relevance to our project**.

Key terms (used throughout)

LLM (large language model)

the neural network (e.g., Llama, Qwen, DeepSeek) that generates text token by token.

Token

a chunk of text (roughly a word-piece) — the unit a model reads and emits.

KV cache (key/value cache)

the model’s stored “attention” state for every past token. Keeping it lets the model avoid recomputing the whole history at each step. It is the main consumer of GPU memory during generation.

Prefill vs. decode

the two phases of inference. *Prefill* reads the whole prompt at once (limited by compute). *Decode* emits one token at a time, each step re-reading the KV cache (limited by memory bandwidth).

TTFT / TPOT

time-to-first-token (how long until the first output appears) and time-per-output-token (the steady-state speed thereafter).

Throughput vs. goodput

tokens/second served overall (throughput) vs. tokens/second that actually meet a latency target (goodput — the number that matters in production).

Prefix caching

reusing the KV cache of a shared beginning (system prompt, tool definitions, conversation history) across requests, so it isn’t recomputed — *if* it hasn’t been evicted.

Eviction

when GPU memory fills, the server discards some cached KV to make room; discarded context must be recomputed if needed again.

Agent / the agent loop

an LLM used in a loop: think → call a tool (search, code execution, etc.) → read the result → think again, for many steps, until a task is done.

Orchestrator vs. serving engine

the *orchestrator* (e.g., LangGraph) decides the agent’s next step; the *serving engine* (e.g., vLLM) actually runs the model. They are separate systems that don’t share state.

Harness / scaffold

the code around the model that turns it into an agent (the prompts, tool wiring, retry logic). The same model scores very differently under different harnesses.

Horizon

how many steps an agent is allowed to take on a task.

Tenancy / interleaving

multiple workloads (e.g., chat and agents) sharing one server, so their requests are mixed together and compete for the same cache.

The locality gap

our central quantity: the difference between how much cached context *could* be reused (with unlimited memory) and how much actually *is* reused (under real, bounded, shared memory).

Cost per verified iteration (“the Cost of Grit”)

our headline metric: the dollar/GPU cost of each agent step, counted against work that actually succeeded — capturing what it costs an agent to “keep trying.”

1. What we are trying to do

This is a **measurement** paper, not a new system or technique. A chatbot answers one prompt and forgets it; an **agent** runs a long loop (think → use a tool → think again) whose memory grows the whole time and is re-read on every step. The server tries to keep that memory in a fast cache (the KV cache), but cache space is scarce and shared with other traffic, so much of it gets evicted and must be recomputed. We measure, on **open infrastructure** (freely available serving software) and **public benchmarks** (openly available task sets), **how much reuse is lost** under realistic mixed traffic and **what that loss costs per unit of useful work**.

Two deliverables:

- **The lead contribution — the “Cost of Grit” curve.** We quantify the gap between available and realized cache reuse (the locality gap) and express it as cost per *verified* agent iteration (cost counted only against steps that contributed to a successful outcome), as a function of how long the agent runs, what caching policy the server uses, and how heavily the server is shared.
- **The artifact contribution — a public dataset + tooling.** We release a recorded stream of mixed chat-and-agent traffic with cost labels, in a standard tracing format, plus the software that collected it. No comparable public dataset exists.

(Internally these are labelled C1 and C3; the full candidate list and the hypotheses are spelled out in plain language in sections 4–6, with their internal labels noted once for cross-reference.)

Why this is defensible. The space of *mechanisms* (clever cache or scheduling tricks) is extremely crowded — many well-funded teams are building them. Far fewer are doing a careful, reproducible *measurement* of what the problem is actually worth, in the open, tied to real task success. That measurement — the bundle of rigor, breadth, open reproducibility, and the released dataset — is the contribution.

2. Where we stand

The literature is fully mapped and fact-checked against live search (the project enforces a strict rule: no citation is used without verification, because earlier drafts contained invented references). The framing has settled from an earlier “find the best recipe” goal onto the measurement goal above. The competitive landscape — including very recent industry results (NVIDIA’s Dynamo serving stack and Artificial Analysis’s AA-AgentPerf benchmark, both summarized in section 7) — has been verified and folded in.

The experiment (“the pilot”) is designed, but two independent critical reviews surfaced **seven design problems that must be fixed before spending money on GPUs** (detailed in section 6) — for example, the core metric is currently defined two inconsistent ways, and the data format cannot yet capture everything needed to compute it. The next concrete step is a co-design meeting with the collaborator who owns the server-internals work, to settle those, then run a small (~\$300, ~1 week) pilot that will either confirm the effect or kill the idea quickly.

Honest assessment: the field is moving fast and the opening is narrowing — mechanism papers are already crowded out, and even individual measurements are adjacent to published work. What remains genuinely open is the specific intersection we target: the locality gap under realistic *mixed* traffic on open infrastructure, tied to verified work, released as a public dataset.

3. The contributions, in full

Contribution	What it is & current standing
Cost-of-Grit curve (lead; internally C1)	Cost/latency per <i>verified</i> agent iteration vs. run length, caching policy, and degree of sharing, on open serving software. The “missing meter” that both serving teams and agent builders would act on. Status: pilot pending the redesign in section 6.
Public mixed-traffic trace + harness (artifact; internally C3)	A released recording of interleaved chat+agent traffic with cost labels and cross-layer timing, plus the collection tool. Status: bundled with the lead contribution.
Cache-aware context-edit policy (secondary; internally C2)	Treating edits to an agent’s context as cost/quality decisions. Status: watching; a natural follow-on paper.
Hint interface (internally C4)	A way for the agent layer to tell the server what to keep/expect. Status: largely pre-empted — NVIDIA’s Dynamo already ships exactly this (see section 7); re-scope to measuring the value of the existing hints, or drop.
Iteration-economics metering (internally C5)	The product-shaped twin of the lead idea; out of scope for this paper.
Harness-conditional benchmarking (internally C6)	Quantifying how much a result depends on the harness vs. the model. Status: background; possible workshop paper.

Already ruled out (the “graveyard”): a generic cache/eviction *mechanism* paper (too crowded); a naive “agents differ from chat” characterization (already common knowledge); “the first public agent trace” as a headline (eroded — survives only in the narrowed, cost-labeled, mixed-traffic form above).

4. The hypotheses (plain language)

Each is a falsifiable claim with a stated “kill criterion” (the result that would make us abandon it). Internal labels in brackets are for cross-reference only.

- **The interleaving hypothesis [H1].** When chat and agent traffic share one server with limited cache memory, the agent’s realized cache reuse falls well below what would be available in isolation, and a measurable part of that drop is caused specifically by the *mixing* of workloads (eviction from competing traffic), as opposed to the agent simply changing its own context. *Kill it if* reuse already collapses even when the agent runs alone (then the story is about the agent, not about sharing).
- **The super-linear cost hypothesis [H2].** Cost per verified iteration grows *faster than linearly* as the agent runs longer, and the shape depends on the caching policy — a structure that a simple “dollars per token” figure hides. *Kill it if* the curve is straight and policy doesn’t matter.
- **The phase-structure hypothesis [H3].** The pauses while an agent waits for tools are not random; they follow a read-then-write rhythm, so a simple “evict the least-recently-used” policy throws away exactly the wrong cache at the wrong moment, and a phase-aware policy would recover much of the loss (shown in simulation, not built). *Kill it if* the pause timing is effectively random.
- **The reproducible-artifact hypothesis [H4].** A small released dataset + tool is enough for others to reproduce the two results above. *Kill it if* a comparable public dataset ships first.

5. The pilot experiment, the metric definitions, and what to fix first

The first experiment is deliberately small and designed to fail fast: run an agent (using **ReAct**, the standard reason-then-act loop — see section 8) on a public tool-use benchmark, once with the server to itself and once sharing it 50/50 with chat traffic, under two caching policies, across 50 tasks × 3 repeats = 600 runs on a single GPU configuration (~1 week, under \$300). The readouts are the available-vs-realized reuse gap, the cost per verified iteration, and the distribution of tool-wait times.

Metric definitions (corrected). *Available reuse* = the fraction of input that could be reused with unlimited cache (computed offline by replay). *Realized reuse* = the fraction actually reused under the real, bounded cache. *The locality gap* = available minus realized, both measured against the **same total**, so it stays between 0 and 1. *Cost per verified task* = total cost (including the cost of failed attempts) per task that succeeded — reported as a curve over run length, to avoid flattering the result by ignoring expensive failures. *Tool-wait time* is the time the agent spends waiting on a tool; note that this is **not** the same as GPU idle time, because under shared traffic the GPU serves other requests during that wait.

Seven design issues to settle before spending GPU budget (from two critical reviews)

1. **The core metric is defined two ways.** The written definition (available minus realized, same denominator) and the code (which divides by reusable tokens and can go negative) disagree. Pick one; make code and docs match; enforce the 0–1 range.
2. **The data format can’t yet compute the answer.** It records token *counts* but not which tokens, the global ordering/timestamps, or which cache entries were evicted and why — all needed to replay “what could have been reused” and to prove the mixing effect. Add those fields. (The same gap means calling it a standards-format trace is premature.)

3. **Sharing and load are tangled together.** Comparing “alone” vs. “50/50 mixed” changes *both* the mix and the total load. Hold the agent’s own work fixed and add chat as a rate-matched background so the effect is attributable to mixing.
4. **The cache size isn’t set.** The very thing that forces eviction (a bounded cache budget) is left unspecified in the configuration. Make it an explicit knob and test more than one setting.
5. **The cost metric has a survivorship bias.** Counting cost only on successes drops the expensive long runs that failed — exactly where the “cost of grit” lives. Count cost across all attempts, per successful task.
6. **Tool-wait time is being mistaken for GPU idle time.** Under shared traffic these differ; measuring idle time needs a server-wide utilization timeline.
7. **The runner doesn’t produce the stated design.** The current loop ignores repeats and run-length and would yield 4 cells, not 600, and reports one global average instead of broken-out, paired results with error bars.

Settle #1, #2, and #5 first — they decide whether the headline number means anything.

6. Annotated literature, part I — how inference works (the foundations)

These are the load-bearing systems papers the project sits on top of. The single organizing idea: generation has two regimes — reading the prompt is compute-limited, writing each token is memory-bandwidth-limited — and almost every technique below either moves work across that line or manages the KV cache (its size, reuse, eviction, precision, or placement).

The model and the attention bottleneck

Attention Is All You Need — Vaswani et al., NeurIPS 2017 · arXiv 1706.03762 ✓

Introduced the Transformer, the architecture behind essentially all modern LLMs. Its core operation, **self-attention** (each token looking at every previous token to decide what matters), is what makes the models powerful — and is also the source of the serving cost, because every past token’s key/value vectors must be kept around (the KV cache) and re-read on each new token.

Why it matters to us: it defines the resource our whole project is about. The KV cache exists because of attention; the locality gap is a statement about how well that cache is managed.

Efficiently Scaling Transformer Inference — Pope et al., MLSys 2023 · arXiv 2211.05102 ✓

The clearest analytical account of *why* inference behaves the way it does: it lays out the “roofline” (a model of when you are limited by compute vs. by memory bandwidth), shows that reading the prompt is compute-bound while generating tokens is bandwidth-bound, and explains how batching and certain attention variants change the picture. Read first to build intuition.

Why it matters to us: it is the mental model behind every measurement we make — if you can’t place an effect on the compute-vs-bandwidth picture, you don’t yet understand it.

FlashAttention (and -2, -3) — Dao et al., NeurIPS 2022 / later · arXiv 2205.14135, 2307.08691, 2407.08608 ✓

A way to compute exact attention without ever writing the huge intermediate score matrix to slow memory — by processing it in tiles that stay in fast on-chip memory. This gave large speedups and made long contexts practical; versions 2 and 3 push it further on newer GPUs. It is now the default attention implementation everywhere.

Why it matters to us: it's a baseline assumption of any modern server we measure; long agent contexts are only feasible because of it.

Multi-Query and Grouped-Query Attention — Shazeer 2019 / Ainslie et al. 2023 · arXiv 1911.02150, 2305.13245 ✓

Two architectural tricks that shrink the KV cache by letting many “query heads” share a smaller number of key/value heads. Multi-Query is the aggressive version (one shared set); Grouped-Query is the now-standard middle ground used by most modern open models. Smaller KV per token means more requests fit in memory.

Why it matters to us: they set how much KV each token costs, which directly affects how soon a shared cache fills and starts evicting — the pressure our interleaving hypothesis is about.

Serving engines, batching, and prefix reuse

Orca: continuous (iteration-level) batching — Yu et al., OSDI 2022 ✓

Before Orca, a server batched requests together and made them all wait for the slowest to finish. Orca introduced **continuous batching**: requests can join and leave the batch at every single token step, so finished requests free their slot immediately and new ones start without delay. This was the single biggest throughput improvement in LLM serving and is now assumed baseline behavior.

Why it matters to us: it is the scheduling baseline our experiments run on; how it interleaves chat and agent requests is part of what creates (or hides) the locality gap.

PagedAttention / vLLM — Kwon et al., SOSP 2023 · arXiv 2309.06180 ✓

vLLM is the de-facto open-source serving engine; its key idea, **PagedAttention**, manages the KV cache like an operating system manages memory — in fixed-size blocks (“pages”) that can be allocated and shared flexibly instead of as one contiguous chunk. This eliminated the memory waste (fragmentation) that previously limited how many requests could run at once, and it enables prefix sharing across requests.

Why it matters to us: it is one of the two engines we measure on, and its paging/eviction behavior is precisely the machinery whose realized vs. available reuse we are quantifying.

Sarathi-Serve: chunked prefill — Agrawal et al., OSDI 2024 · arXiv 2403.02310 ✓

Reading a long prompt (prefill) is compute-heavy and, if done all at once, stalls the steady stream of token generation (decode) for other users. **Chunked prefill** breaks a long prompt into pieces and interleaves them with ongoing decoding, so one user's big prompt doesn't freeze everyone else — trading a little first-token latency for much steadier overall service.

Why it matters to us: agents re-trigger prefill constantly (every loop step re-reads a grown context), so how the engine chunks and schedules prefill directly shapes agent cost and latency.

SGLang / RadixAttention — Zheng et al., NeurIPS 2024 · arXiv 2312.07104 ✓

SGLang is the other major open serving engine. Its key idea, **RadixAttention**, stores cached prefixes in a radix tree (a tree keyed by shared text prefixes) so that *any* requests sharing a beginning — system prompts, few-shot examples, conversation history, branching agent states — automatically reuse each other's KV cache, not just exact repeats. It also speeds up structured (e.g., JSON) outputs.

Why it matters to us: this is the most directly relevant prior system — it is the best case for prefix reuse, so it sets the “available reuse” ceiling our gap measures against. We run it as one of our two engines.

AlpaServe — Li et al., OSDI 2023 · arXiv 2302.11665 ✓

Shows how to place and share multiple models across GPUs to handle bursty, unpredictable demand efficiently (statistical multiplexing) using model parallelism. Less central to single-model agent serving, but part of the canon on serving under variable load.

Why it matters to us: background for the “mixed, bursty traffic” conditions our interleaving hypothesis assumes.

Splitting the two phases apart (disaggregation)

DistServe / Splitwise / Mooncake — OSDI 2024 / ISCA 2024 / 2024 · arXiv 2401.09670, 2311.18677, 2407.00079 ✓

A family of systems built on one insight: the compute-bound prefill phase and the bandwidth-bound decode phase fight each other when run on the same GPU, so **disaggregation** runs them on separate hardware tuned for each. **DistServe** establishes the idea for latency targets; **Splitwise** splits across cheaper/greener machine types; **Mooncake** (the stack behind the Kimi assistant) builds the whole serving system around the KV cache as the central object, including moving it across machines.

Why it matters to us: disaggregation is fundamentally about meeting latency targets efficiently — the same goal as our cost-per-verified-iteration. Agent loops re-trigger prefill constantly, making the prefill/decode tension (and where the KV lives) central to our workload.

Managing the KV cache: compression, eviction, reuse

Prompt Cache — Gim et al., MLSys 2024 · arXiv 2311.04934 ✓

Precomputes and reuses the KV cache of reusable text snippets (e.g., a long document or instruction block) across requests in a modular way, even when they appear at different positions — a more general form of prefix reuse.

Why it matters to us: another point on the “how much can be reused” spectrum that defines our available-reuse ceiling.

StreamingLLM (attention sinks) & H2O (heavy-hitter eviction) — Xiao et al., ICLR 2024 / Zhang et al., NeurIPS 2023 · arXiv 2309.17453, 2306.14048 ✓

Two influential ways to *drop* KV when memory is tight while keeping quality. **StreamingLLM** found that keeping a few initial “sink” tokens plus a recent window is enough for stable infinite-length streaming. **H2O** keeps the tokens that have historically received the most attention (“heavy hitters”) and evicts the rest.

Why it matters to us: these are the eviction policies whose behavior our realized-reuse measurement captures — and whose simple heuristics our phase-structure hypothesis argues are wrong for agents.

KIVI — Liu et al., 2024 · arXiv 2402.02750 ✓

Compresses the KV cache by storing it at very low precision (2-bit) without retraining, roughly quadrupling how much context fits in memory at a small quality cost.

Why it matters to us: a different lever (precision, not eviction) on the same scarce resource; relevant as a knob that changes effective cache size.

FlashMemory-DeepSeek-V4 / Lookahead Sparse Attention — Tencent et al., 2026 · arXiv 2606.09079 ✓ · preliminary/suspended

A recent, learned approach to KV management: a small “Neural Memory Indexer” periodically **predicts which cached chunks the model will actually need next** and keeps only those, shrinking the cache to ~13.5% of full size while holding accuracy. Notably, the predictor is trained on its own (as a retrieval model) without loading the huge backbone model. It is built into the DeepSeek-V4 architecture and is an early, now-suspended project, so its numbers are preliminary.

Why it matters to us: it is a *learned* version of exactly what our phase-structure hypothesis proposes (predict which KV matters), which both validates the premise and confirms the mechanism is being actively pursued — pushing us toward measuring the gap rather than building the predictor. Important distinction: this works *within a single long request*; our project is about *across* mixed requests on a shared cache, so it does not pre-empt the lead measurement.

Spreading work across GPUs, and shrinking the model (background)

Parallelism: Megatron-LM, GPipe, ZeRO, Ring Attention — arXiv 1909.08053, 1811.06965, 1910.02054, 2310.01889 ✓

The standard ways to split a model or its work across many GPUs: **tensor parallelism** (Megatron — split each layer’s math), **pipeline parallelism** (GPipe — split the layers into stages), **sharding** (ZeRO — split the model’s parameters/optimizer state, the basis of today’s FSDP training), and **sequence/context parallelism** (Ring Attention — split a very long input across GPUs). These come from the training world but the concepts carry over to inference.

Why it matters to us: context for how large models are actually served; we hold these fixed and don’t vary them, but they bound the hardware setting.

Quantization: LLM.int8(), GPTQ, SmoothQuant, AWQ — arXiv 2208.07339, 2210.17323, 2211.10438, 2306.00978 ✓

Techniques to run models in lower numerical precision (8-bit or 4-bit) to save memory and increase speed, while preserving accuracy — differing in how they handle the rare large values that make naive quantization fail. AWQ and GPTQ are the common open-weight choices.

Why it matters to us: our chosen model is served quantized to fit one GPU; quantization is part of the fixed setup, and it changes how much memory is left for the KV cache.

Speculative decoding: Leviathan/Chen, Medusa, EAGLE, Lookahead — arXiv 2211.17192, 2302.01318, 2401.10774, 2401.15077, 2402.02057 ✓

A family of speedups that let a model generate several tokens per step instead of one: a cheap “draft” proposes multiple tokens and the real model verifies them in a single pass, accepting the correct prefix. **Medusa** and **EAGLE** avoid a separate draft model by adding lightweight prediction heads; **Lookahead** needs no draft at all. It works because verifying K tokens costs about the same as generating one (the memory wall).

Why it matters to us: a serving knob that changes per-token cost and latency; relevant as a potential confound to control for, and as part of the cost model.

7. Annotated literature, part II — agents, harnesses, and orchestration

ReAct — Yao et al., ICLR 2023 · arXiv 2210.03629 ✓

The foundational agent pattern: interleave **reasoning** (the model thinking out loud) with **actions** (calling tools and observing results), in a loop. Almost every coding/tool agent today is a descendant of this. It is the reason an agent’s context grows step by step and is re-read repeatedly — the behavior that stresses the cache.

Why it matters to us: it is the exact agent loop our pilot runs; the “think → tool → think” structure is what produces the tool-wait pauses and the growing shared prefix we measure.

Reflexion, Toolformer, ReWOO, Tree of Thoughts — arXiv 2303.11366, 2302.04761, 2305.18323, 2305.10601



Variations on the agent loop. **Reflexion** has the agent critique its own failed attempts and remember the lessons. **Toolformer** teaches a model when to call tools. **ReWOO** separates planning the whole tool-use plan up front from executing it (reducing back-and-forth). **Tree of Thoughts** explores multiple reasoning branches instead of one. Each changes the shape of the context and the number of steps.

Why it matters to us: they define the space of agent behaviors whose serving cost differs; ReWOO's plan-then-execute, for example, creates a different reuse pattern than ReAct, a natural axis to expand into after the pilot.

Model Context Protocol (MCP) — Anthropic, late 2024 · open standard ✓

An emerging open standard for how agents connect to tools and data sources (a common “plug” so any agent can use any compliant tool). It is rapidly becoming the default integration layer for agentic systems.

Why it matters to us: it standardizes the tool calls that create the pauses in the agent loop; it's the interface layer where “the orchestrator and the server don't share state” — the gap our framing centers on.

Orchestration frameworks & the harness concept — LangGraph, CrewAI, OpenAI Agents SDK (systems, not papers)

The **orchestrator** is the software that drives the agent loop — deciding the next step, managing memory, calling tools. It is separate from the **serving engine** that runs the model, and the two don't share internal state. The **harness** (or scaffold) is this surrounding code; crucially, the *same* model can score very differently depending on its harness (“home-field advantage”), which is a major source of unfair comparisons in agent evaluation.

Why it matters to us: the orchestrator/server split is the root cause we study — because neither side sees the other, the server must *guess* what KV to keep instead of being *told*. And because the harness can confound results, isolating model effects from serving effects is half our methodological contribution.

Compiling Agentic Workflows into LLM Weights — Dennis et al., Univ. of Melbourne · arXiv 2605.22502 ✓

Shows that a recurring, procedural agent workflow can be “baked into” the model's weights, reaching near-frontier quality at roughly one-hundredth the cost — i.e., for routine tasks, you may not need the expensive live agent loop at all (“internalize” instead of “externalize”).

Why it matters to us: it defines the boundary of our scope. We deliberately target the *non-compileable* workload — open-ended, long-horizon, novel-each-time tasks — because that is exactly where the live loop (and its serving cost) is unavoidable.

8. Annotated literature, part III — the agentic-serving frontier (our direct competition)

Workload characterization (closest prior work)

Agentic AI Workload Characteristics — UIUC / Gimlet Labs / Intel · arXiv 2605.26297 ✓

Profiles real agent runs across several benchmarks and finds that agent serving is dominated by token generation (decode) with **very high potential cache reuse**, and that tool use follows a temporal “read/explore then execute/write” structure.

Why it matters to us: this is the “high available reuse” end of our story — it establishes how much *could* be reused, which our work contrasts against what is actually realized under sharing. Its read/write phase finding is the basis of our phase-structure hypothesis.

Sutradhara — Microsoft Research / M365 · arXiv 2601.12967 ✓

Studies agent serving at production scale (using synthetic-but-realistic requests, not real customer traffic) and finds the opposite-seeming result: cache hit rates **collapse** despite the high potential reuse, and tool calls account for 30–85% of the time to first token. Importantly, it attributes the collapse to churn and eviction *within* each agent request, not specifically to multiple workloads sharing the server.

Why it matters to us: this is the “realized collapse” end — and it defines our precise opening. Because Sutradhara is synthetic and attributes collapse to within-request churn, **nobody has cleanly isolated the effect of multi-workload sharing on open infrastructure**. That isolation is our interleaving hypothesis.

ServeGen — Alibaba / Peking Univ., NSDI 2026 · arXiv 2505.09999 ✓

A large-scale characterization of real production inference (billions of requests) plus an open-source **workload generator** to reproduce realistic traffic patterns — but covering general language/multimodal/reasoning traffic, **not** agent tool-loops, and releasing a generator rather than a raw recording.

Why it matters to us: it is why we must state our novelty precisely: a public *general* serving generator exists, so our claim is the absence of a public *agentic/mixed, cost-labeled* trace specifically.

A CPU-Centric Perspective on Agentic AI — Georgia Tech / Intel · arXiv 2511.00739 ✓

Points out that for agents, a great deal of time and energy goes into the *tool* processing that happens on the regular CPU (not the GPU) — host-side work can dominate latency.

Why it matters to us: reinforces that tool-wait time is real and significant, and warns us to attribute time correctly (CPU tool work vs. GPU idle vs. eviction) — directly relevant to the “tool-wait ≠ GPU-idle” correction in section 5.

Cache & scheduling mechanisms for agents (*this space is crowded → a mechanism paper is not viable*)

Continuum / CacheTTL — Berkeley / Sky · arXiv 2511.02230 ✓

Keeps an agent's cache alive across short tool-call pauses by giving cached entries a time-to-live (a grace period before eviction), rather than evicting them the moment memory is needed — recognizing that the agent will be back shortly.

Why it matters to us: this is the closest mechanism to the “retain during tool call” policy we test as one arm of the pilot; we measure its value rather than propose it.

KVFlow / KVCOMM / Helium / Autellix — arXiv 2507.07400, 2510.12872, 2603.16104, 2502.13965 ✓

Four representative agent-serving systems. **KVFlow** evicts based on a graph of the agent's future steps (so it keeps what will be needed soon). **KVCOMM** shares cache across multiple cooperating agents. **Helium** treats an agent workflow like a database query plan and schedules the model calls accordingly. **Autellix** is a serving engine that treats “agents as programs” with their own scheduling needs.

Why it matters to us: collectively they prove the mechanism space is saturated — which is exactly why we do measurement, not mechanism. They are also the prior art our results would inform (and be compared against).

SparseX / TokenDance / ScaleSim / Aragog / AWO / CONCUR — arXiv 2606.01751, 2604.03143, 2601.21473, 2511.20975, 2601.22037, 2601.22705 ✓

The 2026 wave: interleaved and multi-agent cache sharing (SparseX, TokenDance, ScaleSim), just-in-time model routing and meta-tool compilation (Aragog, AWO), and congestion-based concurrency control for agent batches (CONCUR, ICML 2026). All recent, all mechanism-side.

Why it matters to us: they confirm the field is converging on this problem fast — the window is closing — and that the differentiated lane is open, reproducible measurement rather than yet another mechanism.

Measurement & cost (the angle adjacent to ours)

Don't Break the Cache — PwC · arXiv 2601.06007 ✓

Quantifies how much money and latency prompt-caching saves for long-horizon agent tasks — but measured at the **provider-API level** (treating the model service as a black box), not inside open serving software.

Why it matters to us: it is the closest cost-measurement work, which is exactly why we must distinguish ours: we measure *inside* open infrastructure, where the cache mechanics are visible and controllable, not through a vendor API.

More with Less; Latency-Reliability-Cost tradeoffs; SWE-EVO — arXiv 2510.16786, 2605.23929, 2512.18470 ✓

More with Less measures cost-per-patch for coding agents (about \$5–8 per correct fix over dozens of turns) from the capability side. **Latency-Reliability-Cost** builds an analytical model of the tradeoffs in agent workflows. **SWE-EVO** is a long-horizon software-evolution benchmark (multi-pull-request tasks) with a partial-credit “Fix Rate” metric.

Why it matters to us: these measure cost from the task/capability side; we measure it from the serving side and tie it to verified success — a complementary, distinct angle. SWE-EVO is a candidate benchmark for the long-horizon regime.

Industry & vendor results (*real and source-checked, but vendor material — motivation only, never quoted as evidence*)

NVIDIA Dynamo — agentic inference — developer.nvidia.com ✓ **real**

NVIDIA’s datacenter serving stack. Two parts are directly relevant. Its **KV-aware router** keeps a global index of which cache blocks live on which GPU and sends each request to the GPU that already holds the most of its context (so multi-turn conversations don’t lose their cache). And it exposes **agentic hints** (`nnext.agent_hints`): the agent layer can *tell* the server a request’s priority, expected output length, and to pre-warm a likely cache, plus a time-to-live to pin a prefix. It reports 85–97% cache hit rates in managed, single-session settings.

Why it matters to us: two consequences. (1) It **pre-empts our hint-interface idea** — the “tell the server what to keep” contract is already shipping, so that idea becomes “measure the value of the existing hints,” not “propose them.” (2) Its 85–97% are *realized* hit rates under its own good routing in *single-session* use — it explicitly does **not** study multi-workload sharing on a bounded cache, which is exactly our open question. (These numbers are vendor figures and motivation only, not evidence.)

AA-AgentPerf — Artificial Analysis · artificialanalysis.ai ✓ **real**

A new hardware benchmark for the agent era. It measures how many concurrent agents a system can serve on **real coding-agent trajectories** (up to 200 steps, contexts up to ~131,000 tokens) while meeting latency targets, with the headline metric being **agents served per megawatt** of power. It compares NVIDIA, AMD, and Google hardware. The underlying task set is kept private (held out), and it does not yet tie results to per-task cost or success.

Why it matters to us: it validates that agent serving on long, real trajectories is the workload of interest — but it measures *hardware capacity* (agents per megawatt), not *cost per verified iteration* tied to task success, so our lead metric stays distinct. Because its dataset is closed, it also **strengthens** our “no public agentic trace” claim.

InferenceMAX is an open, continuously-updated benchmark of inference cost-per-token and throughput across hardware — but for general serving, not agents. **NemoClaw / OpenShell** are NVIDIA’s blueprints and sandboxed runtime for building autonomous agents.

Why it matters to us: context for the broader stack and the “cost per token” framing we deliberately move beyond (toward cost per *verified* outcome); NemoClaw is part of the product world the internship sits in, distinct from this open-measurement paper.

9. What we might be missing (threats to the result)

1. **Model vs. system confusion (the top risk).** Many quantities — how efficiently an agent uses tools, how long it reasons, whether it succeeds — depend on the *model*, not the server. If we credit those to a serving knob, the result is invalid. We hold the model fixed and vary only serving settings, and explicitly bound which effects are model-driven.
2. **Prove the sharing effect; don’t assume it.** Because prior work attributes cache collapse to within-request churn, our whole contribution rests on cleanly isolating the sharing effect — hence the rate-matched design in section 5.
3. **Statistical rigor.** The kill criteria depend on the effect exceeding noise. Decide in advance what effect size counts as meaningful, budget enough repeats, and report paired comparisons with error bars.
4. **Benchmark licensing and freshness.** Confirm each benchmark’s license before release; prefer contamination-resistant sets; pin and record exact versions.
5. **Reproducibility and release hygiene.** Target the conference’s artifact-evaluation badge: pin software versions, random seeds, and environment; license the data and the code; ensure the released trace contains no private content and document exactly how it was collected.
6. **Don’t cite vendor headline numbers as evidence.** Use Dynamo’s and AA-AgentPerf’s figures as motivation; measure the claims ourselves on open infrastructure.
7. **Fast-follower risk.** The area is hot; a dated public posting plus the released artifact protect priority better than a slow, perfect paper.

10. Immediate next actions

- **Co-design meeting:** settle the seven design issues in section 5 — lead with the metric definition, the data-format fields, and the cost-survivorship fix; confirm the measurement framing is the lead; agree who does what; decide author order in writing.
- **Redesign** the metric and data format accordingly, then run the small pilot and check it against the kill criteria before scaling up.
- **Decide the hint-interface idea’s fate** — re-scope to measuring the value of the existing (Dynamo) hints on open infrastructure, or drop it.
- **Reconcile** the old planning notes to one venue and one lead idea.

- **Resolve** the still-unverified references and locate the AA-AgentPerf primary source before any of them enter a draft.

Still unverified — do not cite until checked: SideQuest (2602.22603); “Agent Memory” (the id 2606.06448 looks wrong; a real “Agent Memory Below the Prompt” may be at 2603.04428 — confirm); AutoLab (and its “+0.43” figure — do not quote); Inside the Scaffold; GoodServe; Self-Harness; Imcache-agent-trace; ThunderAgent; Agentix@NSDI.